

Block Ciphers: SPNs, Feistel Networks, and DES

CS 171: Cryptography

Sanjam Garg

September 22, 2026

Overview

- 1 Block Ciphers
- 2 Substitution-Permutation Networks
- 3 Feistel Networks
- 4 The Data Encryption Standard
- 5 Upgrading DES
- 6 Summary

Block Ciphers: Recall

Keyed permutation

$F : \{0, 1\}^n \times \{0, 1\}^\ell \rightarrow \{0, 1\}^\ell$, where n is the **key length** and ℓ is the **block length**.
For every key k , the map $F_k(\cdot) = F(k, \cdot)$ is a permutation of $\{0, 1\}^\ell$.

Security

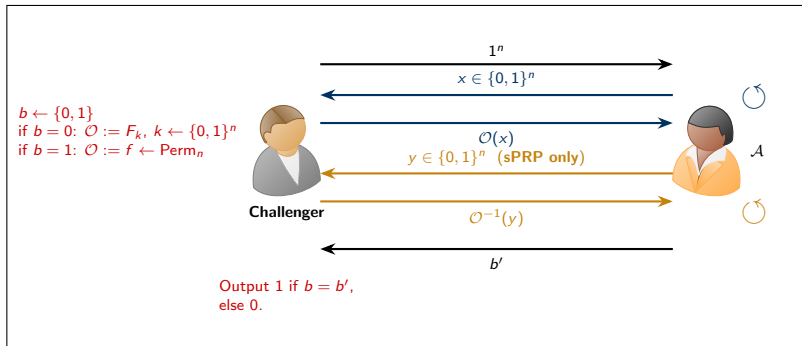
F_k (for uniform k) should be **indistinguishable** from a uniform permutation over $\{0, 1\}^\ell$. Typically we want *strong* security (the distinguisher gets both F_k and F_k^{-1}).

Concrete security

For a key of length n , security is desired against any attacker running in time $\approx 2^n$.

PRP_{A,F} and sPRP_{A,F}: Challenger vs. Adversary

PRP_{A,F}(n) / sPRP_{A,F}(n)



Remark

In PRP_{A,F}, $\mathcal{A}$ may only evaluate $\mathcal{O}$. In sPRP_{A,F}, it may also invert it: F_k^{-1} in one world, f^{-1} in the other.

The Challenge Involved

F_k should be indistinguishable from a uniform permutation over $\{0, 1\}^\ell$.

Avalanche effect

If inputs x and x' differ in *one* bit, what relation between $F_k(x)$ and $F_k(x')$ can we expect?

- **How many** bits do we expect to change? ($\approx$ half of them: $\ell/2$.)
- **Which** bits do we expect to change? (No pattern — each output bit flips with probability $\approx \frac{1}{2}$.)

x	0	1	1	0	1	0	0	1	1	1	0	0	1	0	1	1
x'	0	1	1	0	1	1	0	1	1	1	0	0	1	0	1	1

$F_k(x)$	1	0	0	1	1	1	0	0	0	1	1	0	1	0	0	1
$F_k(x')$	0	0	1	1	0	1	1	0	0	0	1	0	0	0	1	0

8 of the 16 bits flip

Confusion–Diffusion

Confusion

A small change in the input should result in a **local random-looking change** in the output (provided by small substitution tables / S-boxes).

Diffusion

A local change in the output should be **propagated to the entire output** (provided by a mixing permutation that spreads bits across the block).

- Confusion alone $\Rightarrow$ a change stays local; diffusion alone $\Rightarrow$ no nonlinearity. Need **both**, iterated.

Design Paradigms

Two main paradigms for building block ciphers:

- **Substitution–permutation networks (SPNs).**
- **Feistel networks.**

Common theme

Both realize confusion (substitution) and diffusion (permutation/mixing) and iterate over multiple **rounds**, each keyed by a sub-key derived from the master key.

We start from the idea of a random permutation and make it practical.

Substitution–Permutation Networks

- Build **random-looking** permutations on **long** inputs from random permutations on **short** inputs.
- E.g. assuming an 8-byte block length,

$$F_k(x) := f_{k_1}(x_1) f_{k_2}(x_2) \cdots f_{k_8}(x_8),$$

where each f_{k_i} is a random permutation on $\{0, 1\}^8$ and x_i is the i -th byte of x .

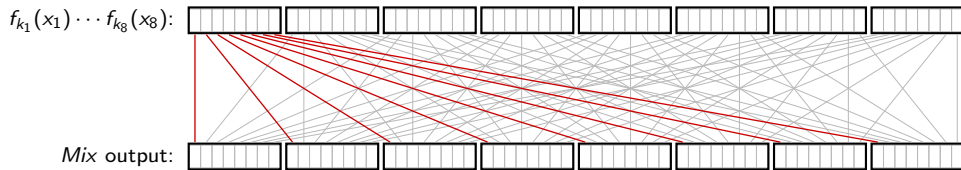
- **Is this a PRP? No!** This has confusion but **no diffusion**.
- **What is the size of the key?** The key *is* the eight permutations, and describing one permutation of $\{0, 1\}^8$ takes $\log_2(256!) \approx 1684$ bits:

$$|k| = 8 \cdot \log_2(256!) \approx 13,500 \text{ bits} \approx 1.7 \text{ KB} \quad \text{for a 64-bit block.}$$

The key for F is already very big!

Adding Mixing

- $F_k(x) := \text{Mix}(f_{k_1}(x_1) f_{k_2}(x_2) \cdots f_{k_8}(x_8))$, where *Mix* is a **public** function.



- *Mix* permutes **bits**, not bytes: the 8 bits of one byte (**red**) land in 8 *different* bytes.
- Every byte's bits are spread the same way — this allows for diffusion, the “propagation of changes”.
- So far, given the key, the construction is invertible and hence a permutation.

Is This a PRP?

input x 

round 1: after $f_{k_1} \cdots f_{k_8}$ 

round 1: after *Mix* 

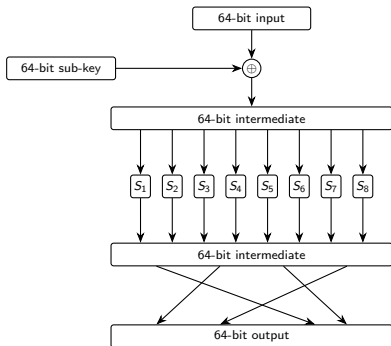
round 2: after $f_{k_1} \cdots f_{k_8}$ 

- The S-box layer keeps the change inside its own byte: 8 of the 64 bits.
- *Mix* scatters those 8 bits, but a *full round* still changes only 8 of the 64 output bits — **not really** a PRP.
- **Fix:** iterate. In round 2 those 8 bits enter 8 different S-boxes, so *all* 64 output bits are affected: the **avalanche effect**.
- **How many rounds?** Carefully decided — and the mixing permutations must be chosen just as carefully.

Making the Key Smaller

- Using random permutations f_{k_i} to start with is **not practical** — the key would be huge.
- Instead, build a round from fixed public components:
 - ① **Key mixing**: set $x := x \oplus k$, where k is the key.
 - ② **Substitution**: set $x := S_1(x_1) \cdots S_8(x_8)$, where x_i is the i -th byte of x .
 - ③ **Permutation**: permute the bits of x to obtain the output.

Add a Mixing Permutation



Public mixing permutation

The wire-crossing layer is a fixed, **public** permutation.

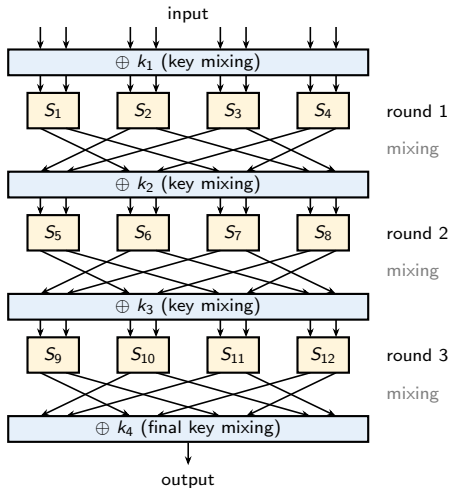
Invertible

Each layer is invertible, so the whole network is an **invertible** permutation.

S-boxes

Replace big random permutations with small fixed **S-boxes** (substitution boxes), then mix.

Repeat with S-boxes



- **Key mixing:** $\oplus$ round sub-key.
- **S-boxes** (substitution boxes): public, invertible.
- **Mixing permutation:** spreads each S-box's output to different S-boxes next round.

Design goal

S-boxes and mixing are designed together to achieve the **avalanche effect**.

Avalanche Effect: Design Principles

- S-boxes and mixing designed **simultaneously**.
 - Small differences should eventually propagate to the entire output.
- **S-boxes**: any 1-bit change in input causes *at least* a 2-bit change in output.
 - Not so easy to ensure!
- **Mixing permutation**: each bit output from a given S-box should feed into a *different* S-box in the next round.

- An r -round SPN has r rounds of:
 - Key mixing,
 - S-boxes,
 - Mixing permutation.
- One additional key mixing is done at the last step.
- **Why?**
- Without the final key mixing, the last round is **invertible** (its S-boxes and mixing permutation are public)!

Invertibility and Strong PRP

- Regardless of the number of rounds, the SPN is **efficient to invert** given the keys (every round step — key mixing, S-boxes, mixing permutation — is invertible).
- S-boxes and mixing permutations are designed so the **avalanche effect applies even when inverting**.

Conclusion

Thus a well-designed SPN gives a **strong PRP**: indistinguishable from a random permutation even under both encryption and decryption queries.

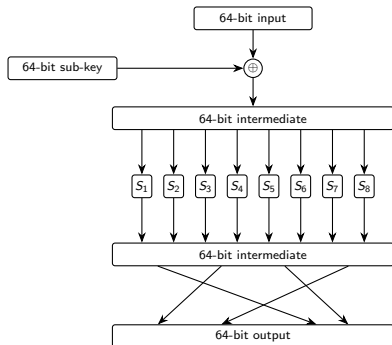
Attacking a 1-Round SPN (no output key mixing)

One-round SPN

$y = F_k(x) = \text{Mix}(S(x \oplus k))$, where S applies the public S-boxes and Mix is the public mixing permutation.

Find k given (x, y) , where $y = F_k(x)$?

- Compute $z := S^{-1}(\text{Mix}^{-1}(y))$ — both layers are **public and invertible**.
- Then $k = x \oplus z$: the key is fully exposed.

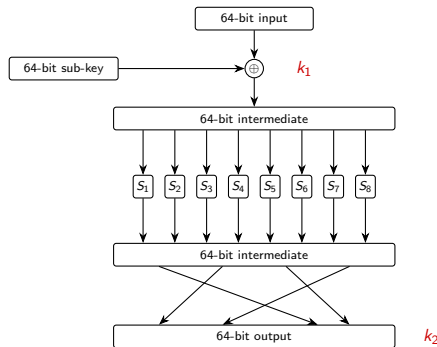


Attacking a 1-Round SPN (with output key mixing)

Find $k = (k_1, k_2)$

$$F_{k_1, k_2}(x) = \text{Mix}(S(x \oplus k_1)) \oplus k_2.$$

- For every k_1 there is a **unique** k_2 (given one pair (x, y)).
- Running time? $\approx 2^{64}$ (brute force over k_1).
- Can we have a better attack? **Same attack: S-box by S-box!**
- Running time? $\approx 8 \cdot 2^8$ — guess only the 8 bits of k_1 feeding one S-box at a time (the matching bits of k_2 follow).



Feistel Networks

- In SPNs, the starting components were **invertible**.
- In Feistel networks, we build **invertible permutations** starting from **non-invertible** components.

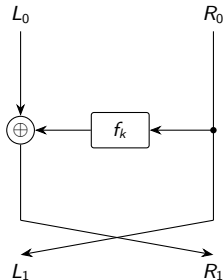
One Feistel round

Split input into halves (L_0, R_0). Using round function f_k :

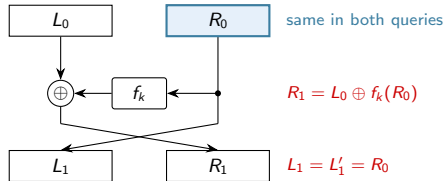
$$L_1 := R_0,$$

$$R_1 := L_0 \oplus f_k(R_0).$$

Invertible even when f_k is **not** invertible.



Feistel Networks: Is One Round Secure?



Is 1 round secure?

Query on (L_0, R_0) and on (L'_0, R_0) — two different left halves, the **same** right half. Can you tell this from a uniform permutation?

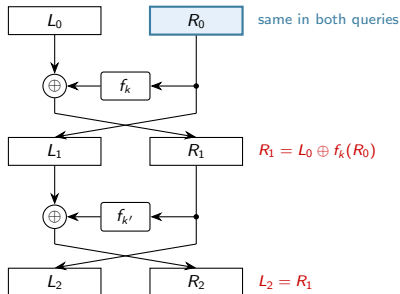
No! The left halves collide

$L_1 = L'_1 = R_0$: the left output half does not depend on the key at all.

And the right halves differ by a known value

$R_1 \oplus R'_1 = (L_0 \oplus f_k(R_0)) \oplus (L'_0 \oplus f_k(R_0)) = L_0 \oplus L'_0$, since the two red terms are equal and **cancel**. A uniform permutation shows neither relation.

Feistel Networks: Are Two Rounds Secure?



Is 2 rounds secure?

Query on (L_0, R_0) and (L'_0, R_0) again, where L_0 and L'_0 differ in *one bit*.

No! The same term cancels

$L_2 = R_1 = L_0 \oplus f_k(R_0)$, so $L_2 \oplus L'_2 = L_0 \oplus L'_0$: the left output halves still differ by exactly the input difference.

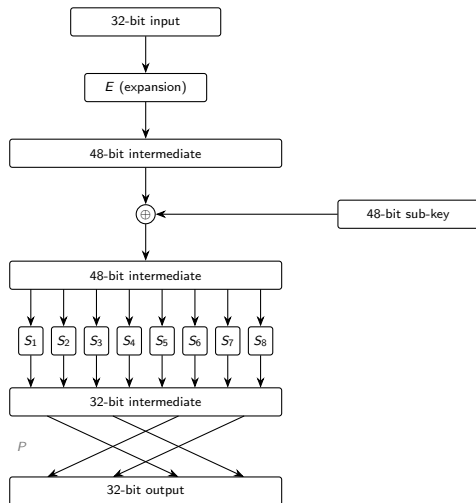
Need 3 or more rounds

Three (or more) rounds with pseudorandom round functions give a pseudorandom permutation (Luby–Rackoff).

The Data Encryption Standard (DES)

- Developed in the 1970s and adopted in 1977.
- **56-bit** keys and **64-bit** block length.
- Attacks in $\approx 2^{56}$ time — too small today; security can be upgraded using **triple DES**.
- A **16-round Feistel network**.
- Uses the same **mangler** function in each round (as the round function).
- The mangler function is essentially an **SPN**.
- Different sub-keys for each round are derived from the master key.

The DES Mangler Function



S-boxes designed so that

- Each S-box is **4-to-1**.
- Changing 1 input bit changes ≥ 2 output bits.

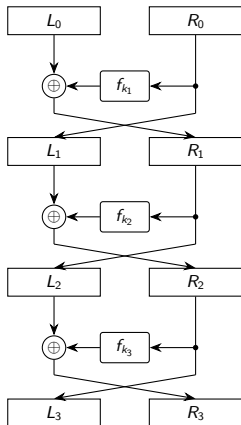
Mixing permutation & E

The 4 output bits of any S-box affect the input to **6 S-boxes** in the next round (diffusion).

Key schedule

Each sub-key takes 48 of the 56 master-key bits (left 24 from the left 28, right 24 from the right 28).

DES Construction



One round

$L_i := R_{i-1}$, $R_i := L_{i-1} \oplus f_{k_i}(R_{i-1})$, where f is the mangler function.

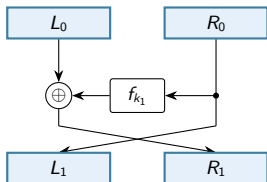
Sub-keys k_i

Each round sub-key k_i is **derived from the 56-bit master key**.

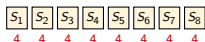
Repeat

Repeated 16 times for the **avalanche effect**!

One-Round DES: Key-Recovery Attack



$$f_{k_1}(R_0) = L_0 \oplus R_1$$



One round, one known pair

Given (L_0, R_0) and the output (L_1, R_1) of **one** Feistel round — can you recover k_1 ?

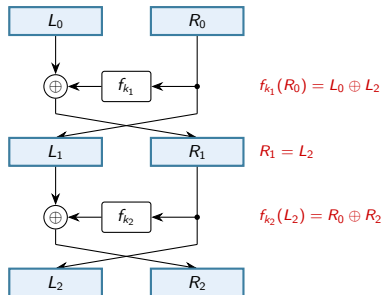
Observation

$R_1 = L_0 \oplus f_{k_1}(R_0)$, so $f_{k_1}(R_0) = L_0 \oplus R_1$: one input/output pair of the mangler is **known**.

Attack: recover k_1 **S-box by S-box**

- Invert the public mixing permutation: each S-box's 4-bit output.
- Each S-box is **4-to-1**: 4 candidates for its 6 sub-key bits.
- In total $4^8 = 2^{16}$ — **far fewer than** 2^{48} .

Two-Round DES: Key-Recovery Attack



Two rounds, one known pair

Given (L_0, R_0) and (L_2, R_2) — can you recover k_1 and k_2 ?

Unrolling

$$\begin{aligned} L_1 &= R_0, & R_1 &= L_0 \oplus f_{k_1}(R_0), \\ L_2 &= R_1, & R_2 &= L_1 \oplus f_{k_2}(R_1). \end{aligned}$$

Reduce to two one-round attacks

- $f_{k_1}(R_0) = L_0 \oplus L_2$ is observable, giving k_1 .
- $f_{k_2}(L_2) = R_0 \oplus R_2$ is observable, giving k_2 .
- Each is the **one-round attack** above, run separately.

More Attacks

- There are attacks better than brute-force key recovery for **three-round** DES.
- **Biham and Shamir** (differential cryptanalysis) gave a $\approx 2^{37}$ -time attack on full DES given $\approx 2^{47}$ chosen plaintexts.
- Considered **not practical**, but it broke the brute-force barrier.

Upgrading Security

Two approaches to a larger key:

- **Modify DES** internally to work with larger keys.
 - Risky and error prone!
- **Build on DES in a black-box manner** — treat DES as a fixed block cipher and compose it.

Attempt 1: Double DES

Construction

$\text{Enc}_{k_1, k_2}(x) = \text{DES}_{k_2}(\text{DES}_{k_1}(x))$, where k_1, k_2 are independent keys (total key length 112 bits).

Does this double the security?

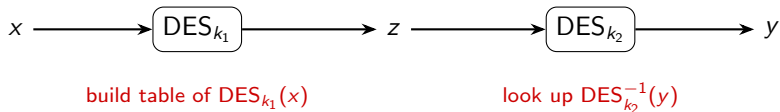
If the best attack on single DES takes 2^n time ($n = 56$), does the best attack on double DES take $\approx 2^{2n}$ time?

- **No!** There is still an attack taking only $\approx 2^n$ time...
- ...but it needs $\approx 2^n$ **memory** (the meet-in-the-middle attack).

The Meet-in-the-Middle Attack

Given input/output pair (x, y) with $y = \text{DES}_{k_2}(\text{DES}_{k_1}(x))$

The midpoint satisfies $\text{DES}_{k_1}(x) = \text{DES}_{k_2}^{-1}(y)$.



- Exhaustively find all (k_1, k_2) such that $\text{DES}_{k_2}^{-1}(y) = \text{DES}_{k_1}(x)$: store all values $\text{DES}_{k_1}(x)$ in a table, then look up $\text{DES}_{k_2}^{-1}(y)$ for each k_2 .
- Assuming random behavior, $\approx 2^n$ candidate pairs (k_1, k_2) survive.
- Test each surviving candidate with another input/output pair.

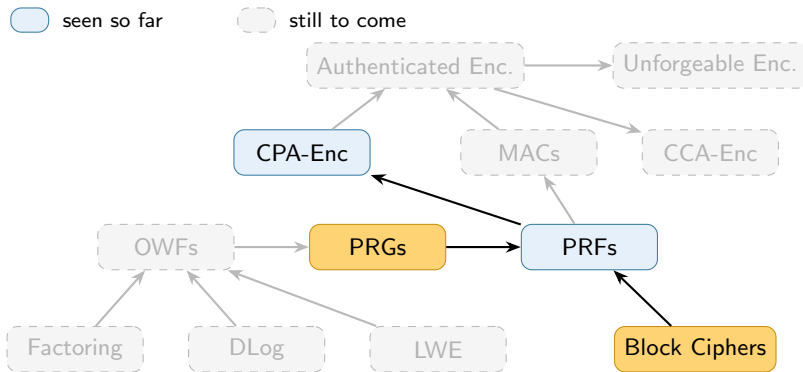
Attempt 2: Triple DES

Construction

$\text{Enc}_{k_1, k_2, k_3}(x) = \text{DES}_{k_3}(\text{DES}_{k_2}^{-1}(\text{DES}_{k_1}(x)))$, with independent keys k_1, k_2, k_3 .

- Best attack takes time $\approx 2^{2n}$ (meet-in-the-middle no longer helps enough).
- Now we also have **AES** (winner announced in 2000).
 - Uses the SPN framework.
 - Will not cover in class.

Too Fragile?



The point

Block ciphers feel ad hoc, but everything we build rests on **well-defined primitives and assumptions**; security reduces down to a few hardness assumptions.

Summary

- **Block ciphers** are keyed permutations; we want the avalanche effect and concrete $\approx 2^n$ security.
- **Confusion** (S-boxes) plus **diffusion** (mixing), iterated over rounds, gives the avalanche effect.
- **SPNs**: each round is key mixing $\rightarrow$ S-boxes $\rightarrow$ mixing permutation, plus a final key mixing; every step is invertible, so a good SPN is a strong PRP. One round is broken S-box by S-box ($\approx 8 \cdot 2^8$).
- **Feistel networks**: an invertible permutation from non-invertible round functions (needs ≥ 3 rounds).
- **DES**: 16-round Feistel, 56-bit key, SPN-style mangler function; broken by brute force ($\approx 2^{56}$) and by differential cryptanalysis.
- **Upgrading**: Double DES fails (meet-in-the-middle, $\approx 2^n$ time / 2^n memory); **Triple DES** ($\approx 2^{2n}$); modern choice is **AES** (an SPN).